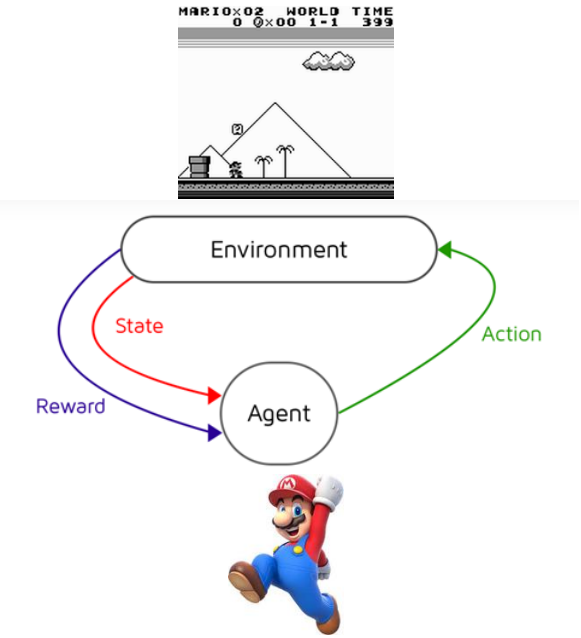
Reinforcement Learning

Fundamentals of Artificial Intelligence

Sequential Decision Making

reinforcement learning (RL):

formalization of sequential decision making (action policy) of software agent interacting with environment



corresponds to search for best (or rather good) action policy to reach a given goal (e.g., win a game)

using learning from examples (data) to guide the search

RL usually more difficult (e.g., non-differentiable as a whole) than supervised learning (which can be seen as “generalized optimization”, often of proxy metric)

Main Elements of RL

goal: find action policy maximizing reward from environment

action policy

- exploration-exploitation trade-off
- e.g., epsilon-greedy: random exploration at small fraction of the time

feedback from environment

- goal-directed, no supervision
- scalar reward signal, cumulative and delayed (credit assignment problem)

Optional Elements of RL

value functions for states or actions: improve efficiency of search in vast action policy space (alternative: direct policy search)

model of environment: (model-free) learning from trial-and-error or (model-based) planning

can be used in different ways:

- simulate experience from model (for learning)
- decision-time planning (e.g., heuristic search or model predictive control)

Markov Decision Process (MDP)

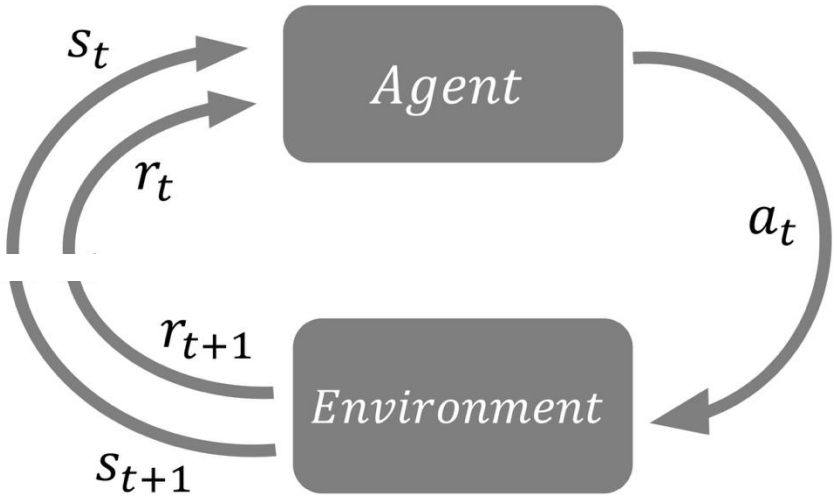
idea: current state includes all information about past

transition probabilities between states describe dynamics of given MDP

action policy: mapping from states to probabilities for selection of different actions

States, Actions, and Rewards

transition probabilities (model of environment): $p(s_{t+1}, r_{t+1} | s_t, a_t)$



reward hypothesis:

- reward as scalar signal
- goal: maximization of expected cumulative sum of received rewards

State and Action Values

state/action value: total amount of expected future reward starting from given state/action (usually with discounting of later steps)

→ indicating long-term desirability of states/actions

main motivation: improve efficiency of search in policy space

(for comparison: evolutionary methods search directly by evaluating entire policies)

State-Value Function

(needed for all states)

on-policy

return

discount rate

$$v_\pi(s_t) = E_\pi\left[\overbrace{\sum_{k=o}^{\infty} \gamma^k r_{t+k+1}}^{\text{return}} \,|s_t\right] = E_\pi[r_{t+1} + \gamma v_\pi(s_{t+1})|s_t]$$

$$= \sum_{a_t} \pi(a_t|s_t) \sum_{s'_{t+1}, r_{t+1}} p(s'_{t+1}, r_{t+1}|s_t, a_t)[r_{t+1} + \gamma v_\pi(s'_{t+1})]$$

policy: probabilitiy to take specific action being in a given state

transition probability (depending on environment) from state s_t to state s'_{t+1} for a given action

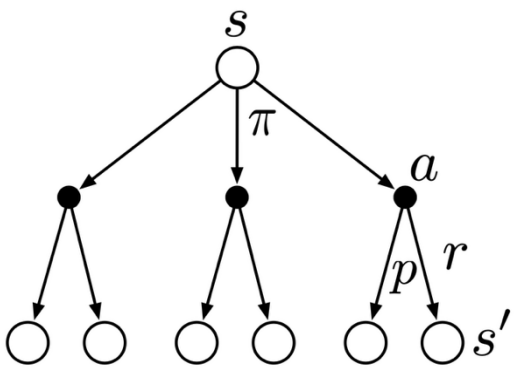
(sweep through entire state space)

Bellman (expectation) equation: recursion

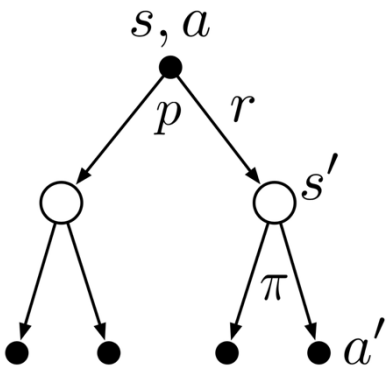
Action-Value Function

$$q_\pi(s_t, a_t) = E_\pi\left[\sum_{k=o}^{\infty} \gamma^k r_{t+k+1} \,|s_t, a_t\right] = E_\pi[r_{t+1} + \gamma q_\pi(s_{t+1}, a_{t+1})|s_t, a_t]$$

$$= \sum_{s'_{t+1}, r_{t+1}} p(s'_{t+1}, r_{t+1}|s_t, a_t)\left[r_{t+1} + \gamma \sum_{a'_{t+1}} \pi(a'_{t+1}|s'_{t+1})\, q_\pi(s'_{t+1}, a'_{t+1})\right]$$



Backup diagram for v_π



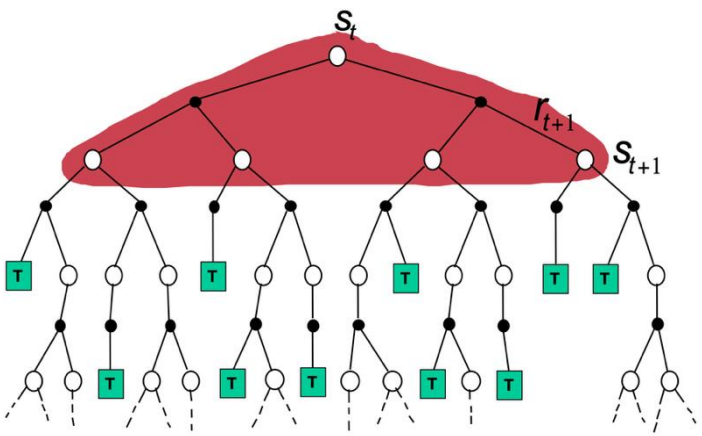
q_π backup diagram

source

Bootstrapping and Sampling

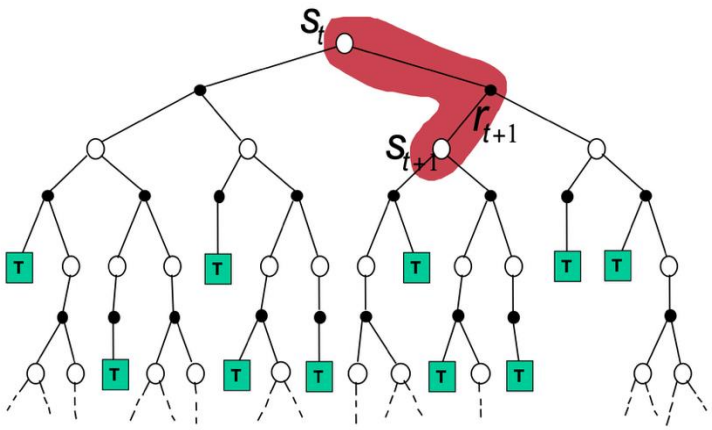
bootstrapping: update estimates of state values based on estimates of values of successor states
sampling: experience of sample sequences (no need for complete knowledge of environment)

Dynamic Programming



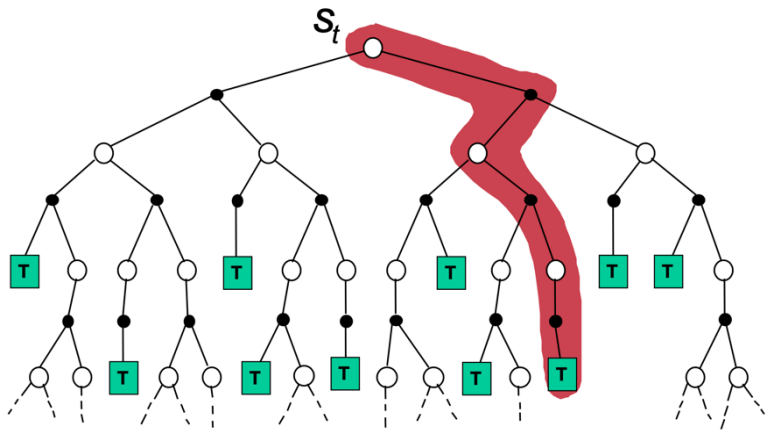
- bootstrapping
- no sampling → model-based (transition probabilities needed)

Temporal Difference (TD) Learning



- bootstrapping
- sampling → model-free

Monte Carlo (MC)



source

- no bootstrapping
- sampling → model-free

Sampling Update Rule

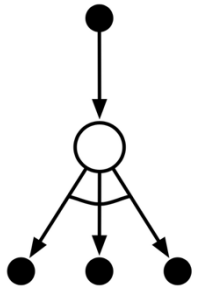
$$\boxed{NewEstimate \leftarrow OldEstimate \; + \; StepSize \left[Target - OldEstimate \right]}$$

MC: $v(s_t) \leftarrow v(s_t) + \eta[\sum_{k=o}^{\infty} \gamma^k r_{t+k+1} - v(s_t)]$

TD: $v(s_t) \leftarrow v(s_t) + \eta[r_{t+1} + \gamma v(s_{t+1}) - v(s_t)]$

bootstrapping

Q-Learning



approximation of action-value function:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]$$

behavior policy (used to collect data) can differ from target policy (greedy) → off-policy

Q stands for quality function: estimate of true action-value function

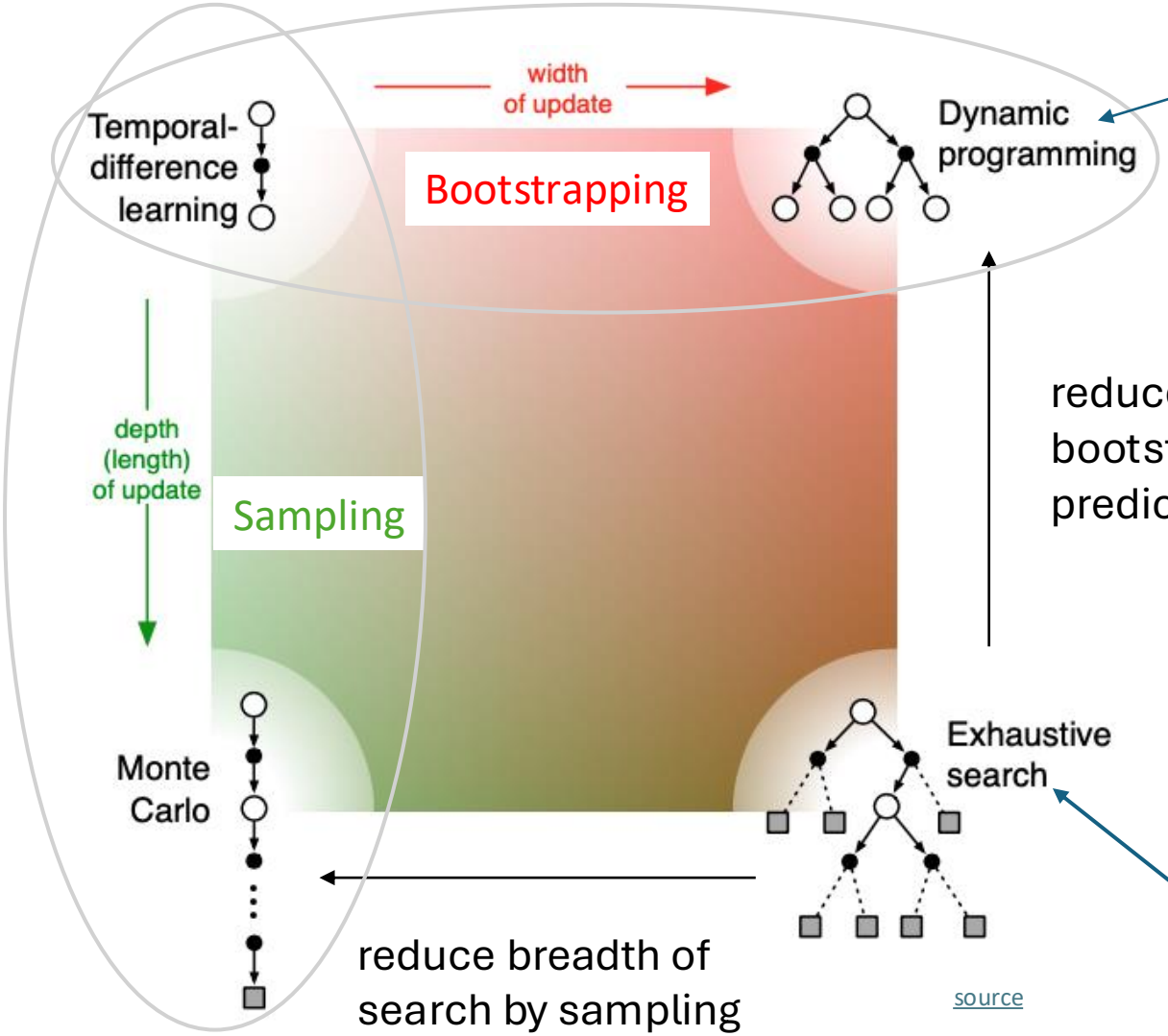
control strategy: choose action with highest Q-value (exploitation)

Update Characteristics

one-step TD

n-step TD

MC



model of environment needed

reduce depth of search by bootstrapping with value function predicting future outcome

reduce breadth of search by sampling actions from policy

very inefficient

source

Limitation of Tabular Methods

tabular methods (calculating values for each state/action) simply memorize observed data

problem with tabular solution methods in practice: large state/action spaces (kind of curse of dimensionality)

→ need for generalization: supervised learning to the rescue

- non-linear function approximation over state/action space
- nowadays often deep learning methods → deep RL

Approximate Solution Methods

online setting: learning while interacting with environment
offline setting: learning from limited data without further interaction with environment

state/action values as parametrized function (instead of table)

- variables/features describing different states
- parameters (e.g., weights in neural network) to be learned

objective function for supervised learning (e.g., squared error loss):

$$\mathcal{L}(\boldsymbol{w}) = \sum_{s} \left(v(s) - \hat{v}(s; \boldsymbol{w}) \right)^2$$

parameters/weights to be optimized

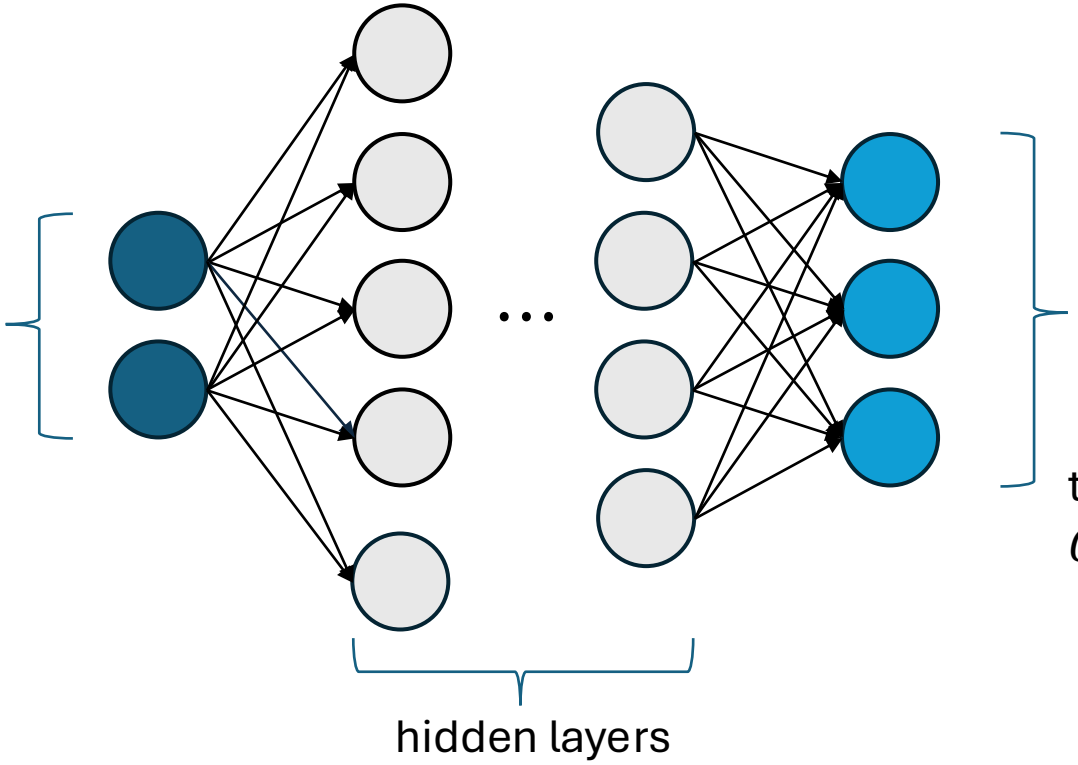
→ RL problem expressed in supervised learning setup

but $v(s)$ still calculated via RL methods (e.g., bootstrapping)

Deep Q-Network (DQN)

idea: approximation of tabular Q-function by neural network

values describing the state



Q-values for the different actions

target:
$Q(s_t, a_t) = r_{t+1} + \gamma \max_a Q(s_{t+1}, a_{t+1})$

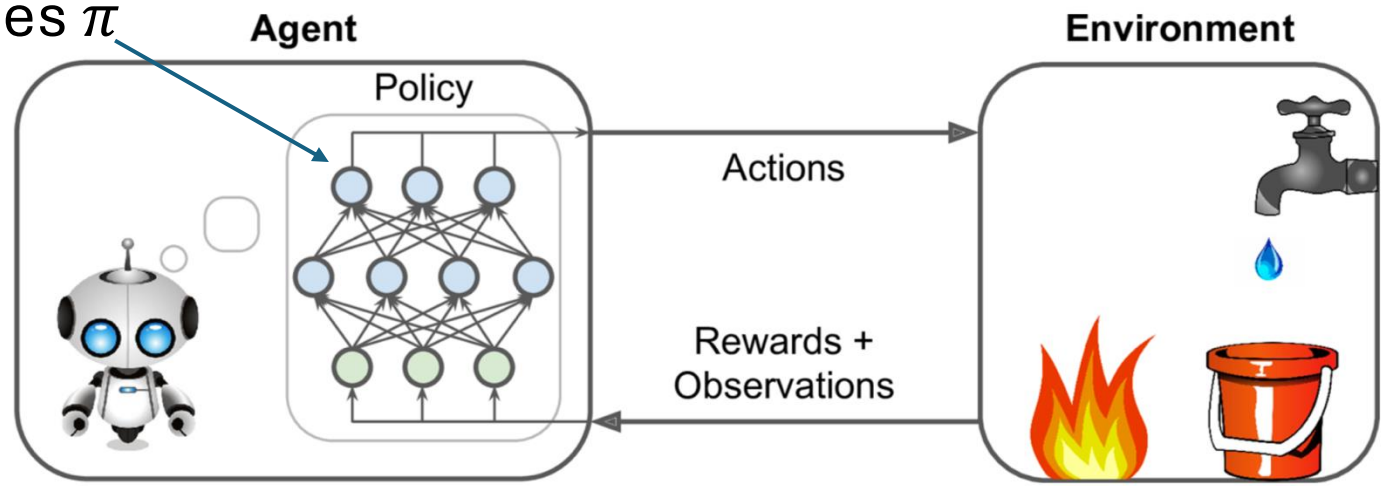
prominent method: Proximal Policy Optimization (PPO)

Direct Policy Search: Policy Gradient Method

neural network with actions as output/target: policy network

action probabilities π

$\log \pi$: score function



observed returns (MC sampling)

$G_t = r_{t+1} + \gamma r_{t+2} + \cdots$

or TD error (actor-critic)

loss: $-\sum_t \log \pi(a_t|s_t; \boldsymbol{w})\, G_t$

gradient ascent
→ actions with higher rewards

choose action according to probability
→ implicit exploration, on-policy method

AlphaGo, AlphaGo Zero

Famous Example of Deep RL: AlphaGo

Monte Carlo tree search (heuristic, lookahead search) for move (i.e., action) selection

→ decision-time planning (advantage: focus on current state rather than full state space)

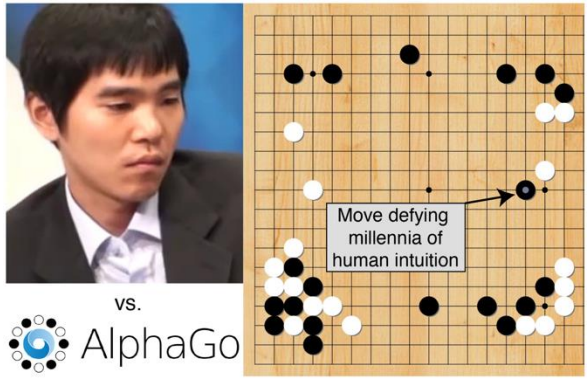
guided by deep convolutional neural networks for both value function and policy estimation

→ improving search efficiency

reduce depth of search tree by evaluating positions with **value function** (predicting outcome from given position → **bootstrapping**)

reduce breath of search tree by **sampling** actions using **policy network** (probability distribution over possible moves in given position)

famous move 37:



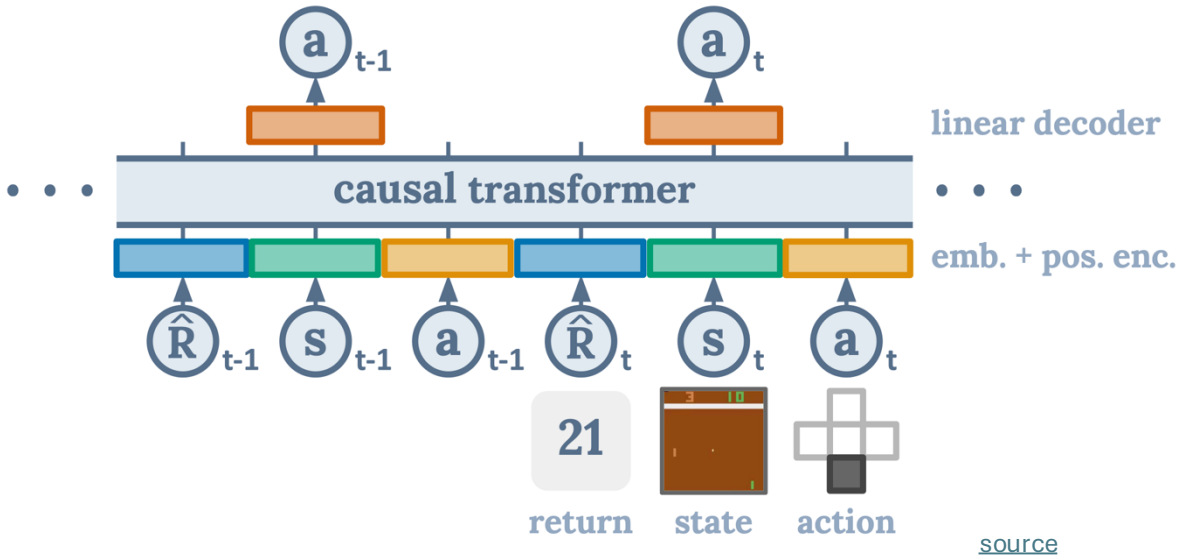
from Max Tegmark

Sequence Modeling for Decisions/Actions

generative: transformer decoder architecture to autoregressively model trajectories

credit assignment directly via self-attention: implicitly forming state-return associations via similarity of query and key vectors (maximizing the dot product)

desired return tokens as prompt for action generation



source

Decision Transformer: conditioning on desired return, past states and actions to generate future actions